

NLP amaliyotida MLOps

Tabiiy tilni qayta ishlash — 16-ma'ruza (yakuniy)

A.A. Abdulali

11:00–12:20 · mavzu №16

10-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarishga uzatish bosqichlari
- 3 API yaratish va dokerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD payplaynlari
- 6 Xulosa, kapstone himoyasi va keyingi yo'l

Takrorlash: o'tgan darsdan ikki savol

1-savol

L15/P15 da hujjat yordamchisi agentini quridik. U sizning noutbuckingizda ishlaydi — foydalanuvchilarga qanday yetkazamiz?

2-savol

Model bir oydan keyin ham bir xil aniqlikda ishlaydimi? Yangi ma'lumot kelsa nima qilamiz?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L15/P15 da hujjat yordamchisi agentini quridik. U sizning noutbudingizda ishlaydi — foydalanuvchilarga qanday yetkazamiz?

2-savol

Model bir oydan keyin ham bir xil aniqlikda ishlaydimi? Yangi ma'lumot kelsa nima qilamiz?

1-javob

Modelni **xizmat** (API) qilib joylashtiramiz va **konteynerga** (Docker) o'raymiz — M4 da boshlagan ishimiz.

2-javob

Yo'q — model **eskiradi** (drift). Monitoring, versiyalash va qayta o'qitish kerak. Bularning hammasi — **MLOps**.

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- ➊ **Tushuntira olasiz:** modelni ishlab chiqarishga uzatish bosqichlari (train → package → serve → monitor).
- ➋ **Tushuntira olasiz:** API yaratish va dokerlashtirish (FastAPI + Docker).
- ➌ **Tushuntira olasiz:** model monitoringi, drift va versiyalash.
- ➍ **Tushuntira olasiz:** CI/CD payplayni va NLP loyihangizni davom ettirish yo'llari.

P15 (agent), M4 P16 (FastAPI/Docker). Tushuncha: API, JSON, model fayllari. Bu dars M4 da amalda qilganingizni **rasmiylashtiradi**.

Bugungi dars rejasi:

Tsikl	Mavzu	Vaqt	Asosiy g'oya
1	Ishlab chiqarishga uzatish bosqichlari	14 daq	train→serve→monitor
2	API va dokerlashtirish	14 daq	FastAPI + Docker
3	Monitoring va versiyalash	14 daq	drift, model registry
4	CI/CD payplaynlari	14 daq	test→build→deploy
	Xulosa, kapstone himoyasi va keyingi yo'l	14 daq	Kurs yakuni

Bu dars — kurs yakuni

Kapstone "O'zbek hujjat yordamchisi" tayyor. Bugun uni ishonchli xizmatga aylantirish (MLOps) va himoya qilishni o'rganamiz.

Bugungi dars uchun muammo: "menda ishlaydi" yetarli emas

Notebook qayerda qiynaladi:

Faqat sizning muhitingiz

Model sizning Kaggle/noutbukungizda ishlaydi; boshqa kompyuterda kutubxona/versiya farqi tufayli ishlamasligi mumkin.

Vaqt bilan eskiradi

Ma'lumot o'zgaradi (drift) $\Rightarrow$ aniqlik tushadi. Hech kim sezmasa, xizmat jimgina buziladi.

Bugungi yechim (MLOps):

- 1 Modelni **paketlash** va **API** qilish.
- 2 **Docker** bilan muhitni qotirib, hamma joyda bir xil ishlatish.
- 3 **Monitoring** + versiyalash + **CI/CD** bilan avtomatlashtirish.

MLOps — modelni ishonchli, takrorlanuvchi va yangilanuvchi xizmatga aylantirish amaliyoti.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarishga uzatish bosqichlari
- 3 API yaratish va dokerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD payplaynlari
- 6 Xulosa, kapstone himoyasi va keyingi yo'l

Intuitsiya: oshxona retsepti emas, restoran

Notebook — bu **retsept** (bir marta, o'zingiz uchun).
Ishlab chiqarish — bu **restoran**: ko'p mijoz, doimiy sifat, tezkor xizmat.

Yo'l: **o'qitish** → **paketlash** (model fayli) → **serve** (API) → **monitoring** (sifatni kuzatish).

Modelni save qilib (m13 save, m14 save_index), keyin yuklab xizmat qilamiz. O'qitish va xizmat — alohida bosqichlar.

Deploy bosqichlari: rasmiy ta'rif

train → package → serve → monitor → (retrain)

bunda: train — modelni o'qitish; package — model fayli + bog'liqliklarni to'plash; serve — API orqali bashorat berish; monitor — sifat va yukni kuzatish; retrain — drift bo'lsa qayta o'qitish (sikl yopiladi). Asosiy g'oya: o'qitish bir marta, **serve** doimiy. Model artefakti (fayl) ikki bosqichni bog'laydi.

Keltirib chiqarish: nega bosqichlarni ajratamiz?

1-qadam. O'qitish qimmat va kam — har so'rovda qayta o'qitib bo'lmaydi.

Keltirib chiqarish: nega bosqichlarni ajratamiz?

1-qadam. O'qitish qimmat va kam — har so'rovda qayta o'qitib bo'lmaydi. **2-qadam.** O'qitilgan modelni **fayl**ga saqlaymiz (artefakt) — bir marta o'qit, ko'p marta yukla.

Keltirib chiqarish: nega bosqichlarni ajratamiz?

1-qadam. O'qitish qimmat va kam — har so'rovda qayta o'qitib bo'lmaydi. **2-qadam.** O'qitilgan modelni **fayl**ga saqlaymiz (artefakt) — bir marta o'qit, ko'p marta yukla. **3-qadam.** Xizmat (serve) faqat faylni yuklab, bashorat beradi — tez, arzon, masshtablanadigan.

Keltirib chiqarish: nega bosqichlarni ajratamiz?

1-qadam. O'qitish qimmat va kam — har so'rovda qayta o'qitib bo'lmaydi. **2-qadam.** O'qitilgan modelni **fayl**ga saqlaymiz (artefakt) — bir marta o'qit, ko'p marta yukla. **3-qadam.** Xizmat (serve) faqat faylni yuklab, bashorat beradi — tez, arzon, masshtablanadigan. **4-qadam.** Monitoring sifatni kuzatadi; pasaysa, retrain bosqichiga qaytadi (sikl).

Xulosa: bosqichlarni ajratish — tezlik, takrorlanuvchanlik va yangilanish imkonini beradi.

Qo'lda misol: yuklashni bir marta qilish

Model fayli 400 MB; yuklash ≈ 2 s, bashorat ≈ 20 ms.

Har so'rovda yuklash: $2000 + 20 = 2020$ ms/so'rov — juda sekin.

Bir marta yuklash (startda): har so'rov ≈ 20 ms — $\sim 100\times$ tezroq.

Shuning uchun model server **ishga tushganda bir marta** yuklanadi (global), so'rovlar uni qayta ishlatadi.

Savol — 60 soniya

Server bir marta yuklangan; har bashorat 20 ms. Bir soniyada nechta so'rovni (taxminan, ketma-ket) bajara oladi?

Sizning vazifangiz: kunlik yuk

Savol — 60 soniya

Server bir marta yuklangan; har bashorat 20 ms. Bir soniyada nechta so'rovni (taxminan, ketma-ket) bajara oladi?

Javob

$1000 \text{ ms} / 20 \text{ ms} = \mathbf{50}$ so'rov/soniya (bitta jarayon, ketma-ket).

Ko'proq kerak bo'lsa — bir nechta nusxa (replica) yoki to'plamlab (batch) ishlash. Bu — masshtablash.

Kod ↔ formula: modelni bir marta yuklash

```
1 # server ishga tushganda BIR MARTA (global)
2 clf = FineTunedClassifier()
3 clf.load("m13_model")           # package -> serve
4
5 def predict(text):              # har so'rov:
6     faqat bashorat
7     return clf.predict(text)
```

Kod → bosqich:

Kod	Bosqich
load	package→serve
predict	serve

load global (bir marta); predict har so'rovda. Bu — M4 P16 dagi naqsh.

Deploy tuzog'i: train va serve muhitlari farqli

Training-serving skew

O'qitishda bir xil, serve da boshqa qayta ishlash (preprocessing) ishlatilsa, natija buziladi. Masalan tokenizator mos kelmasa.

Yechim: bir xil preprocessing kodi (m01) ikki bosqichda; model bilan birga saqlash/versiyalash.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarishga uzatish bosqichlari
- 3 API yaratish va dokerlashtirish**
- 4 Model monitoringi va versiyalash
- 5 CI/CD payplaynlari
- 6 Xulosa, kapstone himoyasi va keyingi yo'l

Intuitsiya: modelni "rozetka"ga ulaymiz va qutiga solamiz

API — modelga standart "rozetka": har kim HTTP so'rov yuborib javob oladi (til/qurilmadan mustaqil).

Docker — modelni va uning butun muhitini bitta **qutiga** (konteyner) solish: hamma joyda aynan bir xil ishlaydi.

FastAPI (M4 P16) — API; Docker — muhitni qotiradi ("menda ishlaydi" muammosini yo'qotadi).

API va konteyner: rasmiy ta'rif

API : request $\rightarrow$ model $\rightarrow$ response; image = base + deps + model + kod

bunda: API — HTTP endpoint (masalan POST /predict); image — Docker tasviri (base OS + kutubxonalar + model fayli + ilova kodi); container — image ning ishlayotgan nusxasi. Image bir marta quriladi, ko'p joyda bir xil ishga tushiriladi — takrorlanuvchanlik kafolati.

Keltirib chiqarish: nega Docker?

1-qadam. Model kutubxona versiyalariga bog'liq (torch, sklearn) — boshqa kompyuterda farq qiladi.

Keltirib chiqarish: nega Docker?

1-qadam. Model kutubxona versiyalariga bog'liq (torch, sklearn) — boshqa kompyuterda farq qiladi. **2-qadam.** Docker base OS + aniq versiyalarni **qotiradi** (requirements.txt).

Keltirib chiqarish: nega Docker?

- 1-qadam.** Model kutubxona versiyalariga bog'liq (torch, sklearn) — boshqa kompyuterda farq qiladi. **2-qadam.** Docker base OS + aniq versiyalarni **qotiradi** (requirements.txt). **3-qadam.** Image ichida model + kod + muhit — bitta birlik.

Keltirib chiqarish: nega Docker?

1-qadam. Model kutubxona versiyalariga bog'liq (torch, sklearn) — boshqa kompyuterda farq qiladi. **2-qadam.** Docker base OS + aniq versiyalarni **qotiradi** (requirements.txt). **3-qadam.** Image ichida model + kod + muhit — bitta birlik. **4-qadam.** Har joyda (bulut, server) **aynan** shu image ishga tushadi — "menda ishlaydi" muammosi yo'qoladi.

Xulosa: API — kirish nuqtasi; Docker — takrorlanuvchi muhit.

Qo'lda misol: Docker image qatlamlari

base (≈ 120 MB) + kutubxonalar (≈ 800 MB) + model (≈ 400 MB).

Jami image ≈ 1320 MB. Kutubxona qatlami o'zgarmasa, qayta qurishda **keshlanadi** — faqat o'zgargan qatlam qayta quriladi.

Shuning uchun requirements ni kod'dan oldin nusxalaymiz — kesh samarali bo'ladi.

Sizning vazifangiz: faqat kod o'zgardi

Savol — 60 soniya

Siz faqat ilova kodini o'zgartirdingiz (kutubxona va model o'sha). Qaysi qatlamlar keshdan olinadi, qaysi biri qayta quriladi?

Sizning vazifangiz: faqat kod o'zgardi

Savol — 60 soniya

Siz faqat ilova kodini o'zgartirdingiz (kutubxona va model o'sha). Qaysi qatlamlar keshdan olinadi, qaysi biri qayta quriladi?

Javob

base va kutubxona qatlamlari **keshdan** (o'zgarmagan); faqat **kod** qatlami qayta quriladi — tez. Shuning uchun Dockerfile da kam o'zgaradigan narsani (deps) yuqoriga, tez o'zgaradigan (kod) ni pastga qo'yamiz.

Kod ↔ bosqich: Dockerfile

```
1 FROM python:3.11-slim          # base
2 COPY requirements.txt .
3 RUN pip install -r requirements.txt # deps (kesh
  )
4 COPY app.py model/ ./          # kod + model
5 EXPOSE 8000
6 CMD ["uvicorn", "app:app", "--host", "0.0.0.0"]
```

Qator → qatlam:

Qator	Qatlam
FROM	base
RUN pip	deps
COPY app	kod

deps kod'dan oldin — kesh samarali. M4 P16 da shu Dockerfile.

Docker tuzog'i: ulkan image va sirlar

Katta va xavfli image

Keraksiz fayllar (datasets, keshlar) image ni shishiradi; API kalitlarini image ichiga yozish — xavfsizlik xatosi.

Yechim: `.dockerignore`; slim base; sirlarni **environment** o'zgaruvchilari orqali (image ichida emas).

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarishga uzatish bosqichlari
- 3 API yaratish va dokerlashtirish
- 4 Model monitoringi va versiyalash**
- 5 CI/CD payplaynlari
- 6 Xulosa, kapstone himoyasi va keyingi yo'l

Intuitsiya: model "qariydi", uni kuzatib turish kerak

Dunyo o'zgaradi: yangi so'zlar, yangi mavzular.

Model o'sha eski ma'lumotda o'qitilgan $\Rightarrow$ aniqligi asta tushadi (**drift**).

Yechim: metrikalarni **kuzatish** (monitoring), pasaysa **ogohlantirish**, kerak bo'lsa yangi **versiyaga** o'tish.

Versiyalash: model registry da v1, v2, ... saqlanadi — xato bo'lsa eski versiyaga **qaytish** (rollback) mumkin.

$$\Delta = M_{\text{baseline}} - M_{\text{joriy}}, \quad \Delta > \tau \Rightarrow \text{ogohlantirish / retrain}$$

bunda: M_{baseline} — dastlabki metrika (masalan F1); M_{joriy} — joriy o'lchov; Δ — pasayish; τ — chegara (masalan 0.10). Metrikalar yo'q bo'lsa (yorliqsiz), **data drift** ni kuzatamiz: kiruvchi matn taqsimoti o'zgardimi (yangi so'zlar ulushi).

Keltirib chiqarish: drift ni qanday aniqlaymiz?

1-qadam. O'qitishdan keyin baseline metrikani qayd etamiz ($F1_0$).

Keltirib chiqarish: drift ni qanday aniqlaymiz?

1-qadam. O'qitishdan keyin baseline metrikani qayd etamiz ($F1_0$). **2-qadam.** Ishlab chiqarishda davriy ravishda joriy metrikani o'lchaymiz (yangi yorliqlangan namunada).

Keltirib chiqarish: drift ni qanday aniqlaymiz?

1-qadam. O'qitishdan keyin baseline metrikani qayd etamiz ($F1_0$). **2-qadam.** Ishlab chiqarishda davriy ravishda joriy metrikani o'lchaymiz (yangi yorliqlangan namunada). **3-qadam.** Farqni hisoblaymiz: $\Delta = F1_0 - F1_{\text{joriy}}$.

Keltirib chiqarish: drift ni qanday aniqlaymiz?

1-qadam. O'qitishdan keyin baseline metrikani qayd etamiz ($F1_0$). **2-qadam.** Ishlab chiqarishda davriy ravishda joriy metrikani o'lchaymiz (yangi yorliqlangan namunada). **3-qadam.** Farqni hisoblaymiz: $\Delta = F1_0 - F1_{\text{joriy}}$. **4-qadam.** Δ chegaradan oshsa (τ), ogohlantirish va qayta o'qitish (retrain) ishga tushadi.

Xulosa: monitoring — "jim buzilish"ni oldini oladi. Endi chegarani qo'lda ko'ramiz.

Qo'lda misol: drift chegarasi

baseline $F1_0 = 0.85$; bir oydan keyin $F1_{\text{joriy}} = 0.70$; chegara $\tau = 0.10$.

Hisob:

$$\Delta = 0.85 - 0.70 = 0.15$$
$$0.15 > 0.10 = \tau \Rightarrow \text{retrain}$$

$\Delta = 0.15$ chegaradan katta $\Rightarrow$ model eskirgan; yangi ma'lumotda qayta o'qitish kerak.

Sizning vazifangiz: ogohlantirish kerakmi?

Savol — 60 soniya

baseline $F1_0 = 0.88$; joriy $F1 = 0.82$; chegara $\tau = 0.10$.

$\Delta = ?$ **Ogohlantirish ishga tushadimi?**

Sizning vazifangiz: ogohlantirish kerakmi?

Savol — 60 soniya

baseline $F1_0 = 0.88$; joriy $F1 = 0.82$; chegara $\tau = 0.10$.

$\Delta = ?$ **Ogohlantirish ishga tushadimi?**

Javob

$\Delta = 0.88 - 0.82 = 0.06 < 0.10 \Rightarrow$ **yo'q**, hozircha normal.

Pasayish bor, ammo chegaradan past — kuzatishda davom etamiz. Chegara (τ) ni juda past qo'ysak, behuda ogohlantirishlar ko'payadi.

Kod ↔ formula: drift tekshiruvi

```
1 BASELINE_F1 = 0.85
2 TAU = 0.10
3
4 def check_drift(current_f1):
5     delta = BASELINE_F1 - current_f1
6     if delta > TAU:
7         return "retrain"          # ogohlantirish
8     return "ok"
9
10 print(check_drift(0.70))          # retrain
```

Kod → formula:

Kod	Formula
delta	Δ
> TAU	$\Delta > \tau$

Davriy ish (cron) bu funktsiyani chaqirib, drift bo'lsa retrain paylaynini boshlaydi.

Monitoring tuzog'i: versiyasiz model

Rollback imkonsiz

Yangi model yomon chiqsa va eski versiya saqlanmagan bo'lsa, ortga qaytib bo'lmaydi — xizmat buzuv qoladi.

Yechim: model registry (v1, v2, ...) + metadata; yomon versiyadan tezda **roll-back**.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarishga uzatish bosqichlari
- 3 API yaratish va dokerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD payplaynlari**
- 6 Xulosa, kapstone himoyasi va keyingi yo'l

Intuitsiya: konveyer — qo'lda emas, avtomatik

Har o'zgarishda qo'lda test/qurish/joylash — sekin va xatoga moyil. **CI/CD** buni **avtomatlashtiradi**: kod o'zgarsa, payplayn o'zi test qiladi, image quradi va joylaydi.

CI (integratsiya): test + build. **CD** (yetkazish): deploy.

Bizning amaliyotlardagi assertlar — CI testlarining asosi: ular avtomatik tekshiriladi.

commit → test → build → deploy (har biri o'tsa, keyingisi)

bunda: commit — kodni yuborish (git); test — avtomatik tekshiruvlar (assertlar, unit testlar); build — Docker image qurish; deploy — yangi image ni serverga chiqarish. Bosqich yiqilsa, payplayn to'xtaydi. Maqsad: har o'zgarish ishonchli, takrorlanuvchi va tez yetkazilsin.

Keltirib chiqarish: nega avtomatlashtirish?

1-qadam. Qo'lda test/deploy — inson xatosi va sekinlik.

Keltirib chiqarish: nega avtomatlashtirish?

1-qadam. Qo'lda test/deploy — inson xatosi va sekinlik. **2-qadam.** Har commit'da testlar **avtomatik** ishga tushadi — buzuvchi kod erta ushlanadi.

Keltirib chiqarish: nega avtomatlashtirish?

1-qadam. Qo'lda test/deploy — inson xatosi va sekinlik. **2-qadam.** Har commit'da testlar **avtomatik** ishga tushadi — buzuvchi kod erta ushlanadi. **3-qadam.** Testlar o'tsa, image avtomatik quriladi (build).

Keltirib chiqarish: nega avtomatlashtirish?

1-qadam. Qo'lda test/deploy — inson xatosi va sekinlik. **2-qadam.** Har commit'da testlar **avtomatik** ishga tushadi — buzuvchi kod erta ushlanadi. **3-qadam.** Testlar o'tsa, image avtomatik quriladi (build). **4-qadam.** Build o'tsa, avtomatik deploy — tez va izchil yetkazish.

Xulosa: CI/CD — sifat darvozalarini avtomatlashtiradi. Kursdagi assertlar shu darvozalarning urug'i.

Qo'lda misol: payplayn bosqichlari

Test 30 s, build 90 s, deploy 20 s; ketma-ket.

Jami ≈ 140 s. Agar test yiqilsa, build va deploy
bajarilmaydi — buzuvchi kod ishlab chiqarishga chiqmaydi.

Tez fikr-mulohaza (feedback): xato bir-
ikki daqiqada ma'lum bo'ladi, kunlar
emas.

Sizning vazifangiz: test yiqildi

Savol — 60 soniya

Payplaynda test bosqichi yiqildi (bitta assert o'tmadi). Keyin nima bo'ladi — build va deploy bajariladimi?

Sizning vazifangiz: test yiqildi

Savol — 60 soniya

Payplaynda test bosqichi yiqildi (bitta assert o'tmadi). Keyin nima bo'ladi — build va deploy bajariladimi?

Javob

Yo'q — payplayn test bosqichida to'xtaydi; build/deploy bajarilmaydi. Buzuq kod ishlab chiqarishga chiqmaydi.

Shuning uchun ishonchli assertlar (kursdagidek) — himoyaning birinchi darvozasi.

Kod ↔ bosqich: CI payplayn (yaml)

```
1 # .github/workflows/ci.yml (soddalashtirilgan)
2 on: [push]
3 jobs:
4   pipeline:
5     steps:
6       - run: pytest                # test
7       - run: docker build .        # build
8       - run: ./deploy.sh           # deploy
```

Qator → bosqich:

Qator	Bosqich
pytest	test
docker	build
build	
deploy.sh	deploy

push bo'lsa avtomatik ishga tushadi; bosqich yiqilsa to'xtaydi.

Faqat deploy avtomatik

Testsiz "avtomatik deploy" — buzuvchi kodni tezroq ishlab chiqarishga chiqaradi. Avtomatlashtirish testsiz xavfli.

Yechim: deploy'dan oldin majburiy test darvozasi; nazorat sinovlari (smoke test) deploy'dan keyin.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarishga uzatish bosqichlari
- 3 API yaratish va dokerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD payplaynlari
- 6 Xulosa, kapstone himoyasi va keyingi yo'l

O'zbek tilida: NLP ekotizimi va ochiq muammolar

1. Mavjud resurslar:

Ochiq modellar paydo bo'lmoqda: UzBERT, Tahrirchi modellari va boshqalar. Ammo ingliz tiliga nisbatan resurslar hamon kam.

Ochiq muammolar

Yorliqlangan ma'lumot kamligi; ishonchli morfologik tahlilchi; domenga xos (huquq, tibbiyot) modellar.

2. Sizning hissangiz:

Kursda qurgan modullaringiz — haqiqiy hissaning boshlanishi.

Yo'nalishlar: o'zbek korpus/benchmark yaratish; ochiq model fine-tune qilib ulashish; domenga xos yordamchilar qurish.

Sintez: notebook va ishlab chiqarish

Jihat	Notebook (tadqiqot)	Ishlab chiqarish (MLOps)
Maqsad	sinash, o'rganish	ishonchli xizmat
Muhit	shaxsiy	Docker (qotirilgan)
Sifat	bir martalik	monitoring + drift
Yangilash	qo'lda	CI/CD + versiyalash

Asosiy g'oya

MLOps modelni **ishonchli**, **takrorlanuvchi**, **yangilanuvchi** xizmatga aylantiradi:
train→package→serve→monitor, Docker, registry, CI/CD.

Seminal manba: Sculley va boshqalar (2015)

Sculley, D., va boshq. (2015). **Hidden Technical Debt in Machine Learning Systems.** *NeurIPS 28*.

Ahamiyati:

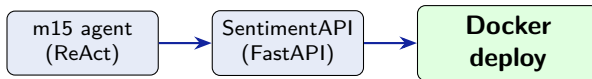
- Model kodi — ML tizimining kichik qismi; atrofidagi infratuzilma katta.
- "Yashirin texnik qarz": monitoringsiz model jimgina eskiradi.
- MLOps amaliyotining nazariy asosi bo'ldi.

Muhokama savoli

O'zbek tilidagi xizmat uchun qaysi "yashirin qarz" eng xavfli — drift, yorliqlangan ma'lumot kamligi, yoki tokenizator nomuvofiqligi? Nega?

- ① ✓ **Tushuntirdik:** deploy bosqichlari (train→package→serve→monitor).
- ② ✓ **Tushuntirdik:** API (FastAPI) va dokerlashtirish (Docker).
- ③ ✓ **Tushuntirdik:** monitoring, drift ($\Delta > \tau$) va versiyalash.
- ④ ✓ **Tushuntirdik:** CI/CD payplayni va o'zbek NLP da keyingi yo'l.

Kapstone himoyasi: "O'zbek hujjat yordamchisi"



Himoyada ko'rsating:

- m15 DocumentAssistantAgent jonli demo (o'zbekcha so'rov).
- SentimentAPI (FastAPI) — POST /predict.
- Docker bilan ishga tushirish (M4 P16).

Tayyorlab keling

16 kun mehnatingiz — bitta ishlaydigan tizim. Modullar (m01–m15), API va MLOps tushunchalarini bog'lab tushuntir-ing.

- ① Sculley, D., va boshq. (2015). *Hidden Technical Debt in ML Systems*. NeurIPS 28.
- ② Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.
- ③ fastapi.tiangolo.com va docs.docker.com — API va konteyner.
- ④ Google / Microsoft MLOps qo'llanmalari (monitoring, CI/CD).

Tabriklaymiz!

16 kunda klassik NLP'dan zamonaviy agentlargacha bordingiz va ishlaydigan o'zbek tili yordamchisini qurdingiz. Endi — bu bilimni real loyihalarda qo'llang va o'zbek NLP ekotizimiga hissa qo'shing.

Ilova S1: model registry va A/B test

Model registry — model versiyalari ombori (v1, v2, ...) metadata bilan (metrika, sana, ma'lumot).
Imkonlar:

- Rollback — yomon versiyadan eskisiga qaytish.
- A/B test — yangi (v2) va eski (v1) modelni real trafikda solishtirish.

bunda: A/B test: trafikning bir qismi v2 ga yo'naltiriladi; metrikalar solishtiriladi; v2 yaxshi bo'lsa to'liq o'tkaziladi.

Ilova S2: smoke test (deploy'dan keyin)

Deploy'dan keyin xizmat haqiqatan ishlayotganini tez tekshiramiz:

```
curl -X POST localhost:8000/predict \  
      -d '{"text": "mahsulot zo'r"}'  
# kutilgan: {"sentiment": "ijobiy", "confidence": ...}
```

bunda: smoke test — eng muhim yo'lining ishlashini tasdiqlovchi minimal sinov; CI/CD da deploy bosqichidan keyin avtomatik bajariladi.